DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

# Motor adaptativo híbrido de Plantia

Cómo se personaliza el catálogo de cursos combinando perfil de cold start, reglas de negocio, comportamiento real y Machine Learning explicable.

Proyecto Plantia

Versión de entrega: motor conectado a PHP, MySQL y la vista de cursos

# Índice

1. Propósito y alcance
2. Motor híbrido
3. Arquitectura y recorrido
4. Datos: perfil, cursos y eventos
5. Reglas de negocio
6. Machine Learning
7. Integración web
8. Ejemplo de decisión
9. Operación y mejoras

# 1. Propósito y alcance

El motor adaptativo ordena el catálogo para que los primeros cursos sean los más pertinentes para cada persona. Todos los cursos publicados permanecen disponibles: el motor no restringe la exploración, únicamente calcula su prioridad.

Responde preguntas como: “¿qué conviene mostrar primero a una persona principiante con poco tiempo?”, “¿qué contenido es más seguro si tiene mascotas?” y “¿qué tema parece interesarle después de abrir varios cursos?”.

> **Principio de diseño:** las reglas explícitas y de seguridad mandan. Machine Learning sólo mejora el orden entre cursos ya permitidos; no puede ignorar una restricción.

# 2. Un motor híbrido

### Reglas

Representan decisiones conocidas: nivel, tiempo disponible, intereses, seguridad con mascotas y conexión limitada. Son transparentes y se pueden explicar.

### Machine Learning

Aprende patrones de interacciones: coincidencias de etiquetas, cercanía de nivel, duración y cursos abiertos o descartados. Es una señal adicional.

Esta combinación soluciona el cold start: desde la primera visita se usan las respuestas del cuestionario. Cuando existen suficientes interacciones, ML añade afinidad sin volver opacas las decisiones.

# 3. Arquitectura y recorrido de una recomendación

**Perfil** → **Catálogo + etiquetas** → **Eventos** → **Reglas** → **ML** → **Orden explicado**

| Componente | Responsabilidad | Implementación |
|---|---|---|
| Recolector | Guarda aperturas e inscripciones. | `adaptive_events` en MySQL. |
| Constructor de contexto | Reúne perfil, etiquetas, cursos, eventos y reglas. | `AdaptiveRecommendationService.php`. |
| Decisor | Evalúa reglas, calcula afinidad y ordena. | `tools/adaptive_engine.py`. |
| Explicador | Devuelve razones y reglas aplicadas. | Campo `reasons` del JSON. |
| Vista | Muestra el orden y “Para ti: …”. | `course-view.js`. |

## Al entrar a Cursos

1. El navegador llama a `api.php?action=courses`.
2. PHP consulta cursos publicados y avance.
3. El servicio adjunta perfil, etiquetas, reglas activas y hasta 200 eventos recientes.
4. PHP envía el contexto en JSON al script Python.
5. Python devuelve puntajes y explicaciones.
6. PHP ordena los cursos y la interfaz dibuja primero los más pertinentes.

## Al abrir o inscribirse

La apertura guarda `opened` y la inscripción guarda `enrolled`. Al volver al catálogo se realiza una consulta nueva y el motor incorpora dichos eventos al orden. No se requiere una recarga manual.

# 4. Datos del motor

## Cold start

Mientras aún no hay comportamiento, se usa el cuestionario: nivel de conocimiento, intereses, objetivo, espacio, región, plantas registradas, mascotas y tiempo disponible. Estos datos residen en `user_profiles` y `user_tags`.

## Contenido

Cada curso aporta nivel, duración estimada y etiquetas de tema, ecosistema, seguridad o formato. Las etiquetas comunes entre persona y curso forman una coincidencia importante.

## Eventos observados

| Evento | Significado | Uso |
|---|---|---|
| `opened` | La persona quiso conocer un curso. | Favorece cursos con etiquetas relacionadas. |
| `enrolled` | Tomó una decisión más fuerte. | Ayuda a complementar en vez de repetir. |
| `dismissed` | Disponible para un control futuro “No me interesa”. | Reduce temas similares. |

> **Privacidad:** se guarda sólo usuario, curso, tipo de evento y fecha. No se registran contraseñas ni contenido privado innecesario.

# 5. Reglas de negocio

Las reglas son datos en `recommendation_rules`, `recommendation_rule_conditions` y `recommendation_rule_actions`. Cada regla tiene prioridad, condiciones y acciones. Por eso se pueden modificar sin cambiar el algoritmo.

```
Regla: poco tiempo
Condición: el usuario tiene la etiqueta time-low
Acciones: +18 a cursos de máximo 90 minutos; -20 a cursos de 240 minutos o más
```

El motor evalúa primero prioridades altas. Una acción puede sumar, restar, excluir o añadir una advertencia. La acción también deja una razón para explicar la recomendación.

# 6. Machine Learning

El motor implementa una regresión logística sin dependencias externas. Aprende con ejemplos `label: 1` (interacción positiva) y `label: 0` (descarte). Sus características incluyen etiquetas del perfil y curso, coincidencias, cercanía de nivel, duración y eventos previos.

Genera una probabilidad de afinidad entre 0 y 1. El cálculo final es:

```
puntaje final = puntaje de reglas + (probabilidad ML × 30)
```

Sin modelo entrenado, la probabilidad es nula y todo funciona sólo con reglas. Por eso ML no bloquea el cold start ni tiene permiso de contradecir seguridad.

## Entrenar

```
python tools/adaptive_engine.py train --input tools/adaptive_training_example.json --model
storage/adaptive-model.json
```

El modelo guarda pesos, sesgo y número de muestras. La exactitud del entrenamiento sirve para detectar errores técnicos; una evaluación real debe usar datos separados de entrenamiento y prueba.

# 7. Ejemplo de decisión

Perfil: principiante, con mascotas, interés por plantas de interior y poco tiempo. Curso: 75 minutos, nivel principiante y etiqueta de seguridad para mascotas.

| Decisión | Puntaje |
|---|---|
| Seguridad con mascotas | +25 |
| Nivel principiante | +25 |
| Poco tiempo | +18 |
| **Total de reglas** | **68** |
| ML: afinidad 0.77 | +23.1 |
| **Puntaje final** | **91.1** |

# 8. Operación, auditoría y mejora

Las decisiones pueden persistirse en JSONL: una línea por evento o decisión. El formato permite auditar el perfil usado, reglas aplicadas, puntaje de reglas, predicción ML y lista final sin sobrescribir el historial.

```
python tools/adaptive_engine.py recommend --input entrada.json --output recomendaciones.json --decisions-log storage/decisions.jsonl
```

La integración PHP posee un respaldo: si Python no puede ejecutarse, el catálogo continúa con su orden normal. Así, el componente adaptativo no bloquea la plataforma.

## Próximas mejoras

- Incorporar el control “No me interesa” y eventos `dismissed`.
- Usar finalización y resultados de cuestionarios como señales de dominio.
- Medir aperturas, inscripciones, finalización, descarte y mejora de desempeño.
- Entrenar con datos reales separados en entrenamiento y prueba.
- Agregar diversidad temática para evitar listas repetitivas.
- Versionar el modelo y revisar sesgos antes de aumentar su complejidad.

> **Conclusión:** es adaptativo porque incorpora nuevas interacciones y vuelve a decidir; es explicable porque conserva las reglas y razones de cada recomendación.

Documento generado para Plantia. Las rutas y nombres corresponden a la implementación actual del proyecto.